

Ingeniería de Software 1

Resumen Completo — Primer Parcial

UBA — Ingeniería Informática

Versión 2 — Ampliada con materiales complementarios

Índice de temas

- 1. Entendimiento del problema (Product Discovery + Design Thinking)
- 2. Outcome vs Output
- 3. Clean Code
- 4. SOLID, buenos y malos diseños
- 5. Agilidad, VUCA, XP y Kanban
- 6. Scrum Framework
- 7. Product Discovery (MVP, Dual Track)
- 8. Historias de Usuario
- 9. Modelo de Dominio
- 10. UML (diagramas de estructura y comportamiento)
- 11. Atributos de Calidad
- 12. Arquitectura de un Sistema (modelo 4+1)
- 13. Patrones de Arquitectura de Software
- 14. RESTful API
- 15. Sprint Planning, Daily y Retros — detalle ampliado
- 16. Headers, Query Strings y Request Bodies en REST

1. Entendimiento del problema

Product Discovery

Disciplina para construir el producto correcto. Los usuarios son quienes saben qué necesitan, por eso definimos métricas para ver qué tanto nuestra solución ataca el punto de dolor.

Convertir una visión en realidad

- ¿El problema es real e importante para el usuario?
- ¿Los clientes valoran nuestra solución? ¿Están dispuestos a pagar?
- El objetivo es enfatizar que entendimos su dolor.

Riesgos al convertir visión en realidad

- Que lo producido no sea valioso ni resuelva un problema real.
- Que no se entienda cómo usar el producto.
- Que la visión sea inviable (tiempo, habilidades, tecnología).
- Que no sea rentable o sostenible (error de planificación).

Prácticas para reducir riesgos

- Entrevistas a clientes.
- Customer journey (pasos del cliente al interactuar con el producto).
- Mapa de empatía: dice, piensa, hace, siente.

Design Thinking

Método para generar ideas, busca eficacia en dar solución a necesidades reales del usuario. Multidisciplinario, innovador y centrado en el usuario.

Conceptos previos

- **Pensamiento divergente:** genera múltiples soluciones, desafía lo convencional.
- **Pensamiento convergente:** evalúa y selecciona la mejor solución.
- **Prototipo:** representación simple e incompleta para testear con usuarios.
- **Mapa de empatía:** piensa, siente, dice, hace, escucha, dolores y necesidades.
- **Shadowing:** observar al usuario sin intervenir para descubrir hábitos inconscientes.
- **Mapa de actores:** cómo se relacionan las entidades en el sistema.
- **Customer Journey:** experiencia completa del usuario al interactuar con el producto.
- **POV (Point of View):** enmarca el problema. Formato: usuario → necesita → problema → porque (insight).
- **Brainstorming:** muchas ideas, juicio diferido, toda idea útil, maximizar, desarrollar y asociar.

Las 5 instancias del Design Thinking

- **1. Empatizar:** investigar. Entrevistas, mapa de empatía, shadowing, mapa de actores, customer journey.
- **2. Definir:** materializar la investigación (POV, patrones de conducta, motivaciones).
- **3. Idear (divergente):** proponer soluciones realistas. Brainstorming.

- **4. Prototipar (convergente):** seleccionar la mejor idea y convertirla en prototipo.
- **5. Testear:** usar el prototipo con el usuario. Cocreación por prueba y error.

2. Outcome vs Output

Lo que importa es el valor que se genera a partir de lo entregado.

Diferencia clave

- **Output (entregable):** lo construido o hecho. Puede sesgar la medición del éxito porque podría no generar impacto real.
- **Outcome (efecto real):** el cambio de comportamiento que el negocio quería generar. Mide el éxito real.

Modelado del problema

- **Recursos:** tiempo, materiales disponibles.
- **Actividades:** tareas preparativas previas al output.
- **Outputs:** lo que concretamos (ej: construir el pozo).
- **Outcomes:** verificar si el output genera valor (ej: que el pozo funcione).
- **Impacto:** cómo contribuye al objetivo general (ej: mejorar calidad de vida del pueblo).

Gestión por outcomes

- Múltiples outputs pueden generar un outcome.
- Múltiples outcomes pueden generar un impacto.

Validación: Hipótesis + Métricas

- **Hipótesis:** "Si construyo X output, entonces Y outcome se alcanzará."
- **Métricas:** evalúan si el outcome se alcanza (agnóstico al output particular).

3. Clean Code

El costo de no tener código mantenible: la productividad decrece con el tiempo. Buscamos: mantenibilidad, simplicidad, flexibilidad, claridad y legibilidad.

Regla del Boy Scout: dejar el código más limpio de lo que lo encontramos.

Nombres

- Significativos y pronunciables.
- Revelan la intención.
- Evitar datos hardcoded sin explicación.

Funciones

- Cortas.
- **Una sola responsabilidad** (SRP).
- Un solo nivel de abstracción (no mezclar alto nivel con sockets).
- Legibles de arriba a abajo como un periódico.
- Pocos argumentos (idealmente 0 o 1).
- En vez de flags (bools como parámetro), usar polimorfismo o nuevas funciones.
- Evitar switches (rompe SRP).
- **No repetir código** (DRY).
- **Principio del mínimo asombro:** el código hace lo que se esperaría.

Comentarios

Son válidos cuando:

- Aclaran temas legales, informativos o explican intención.
- Advierten de consecuencias.
- TODO.
- Amplían info que no está ya descrita en el código.

No deben ser:

- Redundantes, erróneos, tipo diario.
- Código comentado (muerto).
- Preferir excepciones antes que mensajes de error.

Tests unitarios (F.I.R.S.T)

- Una sola cosa por test (único assert por caso).
- **Fast:** rápidos, para ejecutarse todo el tiempo.
- **Independent:** no dependen entre sí.
- **Repeatable:** misma entrada → mismo resultado.
- **Self-validating:** no requiere leer logs para saber si falló.

- **Timely:** oportunos, antes o durante la implementación (TDD).

4. SOLID, buenos y malos diseños

Análisis vs Diseño

- **Análisis:** modela qué necesitamos.
- **Diseño:** modela cómo resolveremos lo necesitado.

Buen diseño

Motivación

- Manejar bien el cambio.
- Lidiar con la complejidad.
- Concretizar rápidamente.

Características

- **Alta cohesión:** cada módulo tiene una responsabilidad bien definida, sus responsabilidades se relacionan entre sí.
- **Bajo acoplamiento:** depende lo menos posible de otros módulos; cambios internos no afectan a otros.

Mal diseño

- **Rigidez:** cualquier cambio requiere ajustes múltiples (alto acoplamiento).
- **Fragilidad:** un cambio rompe funcionalidades aparentemente no relacionadas (mala encapsulación).
- **Inmovilidad:** dependiente del resto del código, no se puede reusar en otros proyectos.
- **Viscosidad:** aplicar hacks o parches en lugar de refactorizar.

Causa principal: dependencias incorrectas entre módulos.

Principios SOLID

S — Single Responsibility Principle

- Una sola responsabilidad.
- Solo porque puedas, no implica que debas.

O — Open/Closed Principle

- Abierto a la extensión, cerrado a la modificación.
- Agregar funcionalidades nuevas sin modificar el código existente.

L — Liskov Substitution Principle

- Las subclases deben poder reemplazar a su clase padre sin romper el código.
- Cada subclase implementa TODOS los comportamientos esperados.
- **Ejemplo:** un Stack NO puede derivar de Vector (indexar stack sería indeseado). Un Vector sí puede implementar un Stack.

I — Interface Segregation Principle

- Interfaces específicas.

- No forzar a una clase a implementar métodos que no usa.

D — Dependency Inversion Principle

- Las clases dependen de abstracciones, no de clases concretas.
- Así se puede cambiar fácilmente la implementación de fondo.
- **Ejemplo:** tomacorriente-lámpara mediante interfaz. El tomacorriente no está anclado a la lámpara: se pueden enchufar otros electrodomésticos.

5. Agilidad, VUCA, XP y Kanban

VUCA

Modelo de gestión de entornos: Volátiles, Inciertos, Complejos, Ambiguos.

Marco Cynefin (5 contextos de toma de decisiones)

- **Claro/simple:** causa-efecto obvia. Soluciones conocidas (mejores prácticas).
- **Complicado:** requiere análisis de expertos. Solución descifrable (buenas prácticas).
- **Complejo:** causa-efecto solo en retrospectiva. Prácticas emergentes, experimentar.
- **Caótico:** sin causa-efecto clara. Acción inmediata, prácticas novedosas.
- **Desordenado:** estado de confusión. Primero diagnosticar en qué dominio estamos.

Agilidad

Enfoque iterativo e incremental para gestionar proyectos. Prioriza flexibilidad, colaboración y entrega continua de valor.

Qué NO es

- No es una metodología paso a paso.
- No implica ir más rápido (sí entrega temprana de valor).
- No implica multitasking.

Qué SÍ es

- Aprender en pasos pequeños.
- Buscar colaboración constante y adaptación al cambio.
- Entrega continua de valor.

Corazón de la agilidad: colabora, entrega, mejora, reflexiona.

Metodologías ágiles

- **Kanban:** tablero visual con tarjetas que se mueven por columnas (Trello, Jira).
- **Scrum:** más estructurada, organiza sprints con roles y ceremonias.

Extreme Programming (XP)

Metodología ágil de desarrollo de software. Se enfoca en mejora continua, colaboración estrecha y entrega frecuente de software funcional.

Se llama "extremo" porque lleva las mejores prácticas al máximo, no las practica ocasionalmente.

6. Scrum Framework

Marco de trabajo ágil basado en sprints (ciclos cortos). El equipo se autoorganiza para entregar incrementos funcionales, usando ceremonias para estructurar el flujo.

Conceptos clave

- **Historia de Usuario:** "Como <rol>, quiero <funcionalidad>, para <beneficio>." No se centra en detalles técnicos. Tiene criterios de aceptación.
- **Stakeholder:** cualquier persona con interés en el producto (usuarios, negocio, externos).
- **Épica:** unidad grande de trabajo, necesidad de alto nivel. Se divide en historias de usuario.
- **Product Backlog:** lista mantenida por el Product Owner. TODO lo que se necesita, priorizado por valor. Dinámico.
- **Sprint Backlog:** subconjunto del product backlog para un sprint específico. Tareas atómicas y estimadas. Fijo durante el sprint.

Roles

- **Product Owner:** define los criterios de aceptación y gestiona el backlog.
- **Scrum Master:** facilita ceremonias, elimina obstáculos, guía al equipo.
- **Development Team:** desarrolladores que implementan. Definen el esfuerzo necesario.

Ceremonias

- **Sprint Planning:** al inicio del sprint. Se eligen historias, se cargan al sprint backlog, se define objetivo.
- **Daily Scrum:** ~15 min diarios. Sincroniza equipo, identifica bloqueos.
- **Sprint Review (DEMO):** último día. Se muestra lo conseguido a stakeholders.
- **Retro:** reflexión final sobre el sprint.
- **Refinement del Backlog:** constante, el PO refina historias prioritarias.
- **Parking Lot:** aborda y documenta impedimentos en cualquier momento.

Estimaciones

No necesitan ser exactas. Miden esfuerzo relativo (complejidad + tamaño), NO tiempo ni complejidad pura. Se usan story points.

- **Planning Poker:** consenso. Cartas Fibonacci, se discuten diferencias hasta acordar.
- **T-shirt Sizes:** estimación rápida con tallas (S, M, L, XL).
- **Velocidad del equipo:** cantidad de story points que completa por sprint.

Métricas de velocidad

- **Burndown Chart:** progreso DEL SPRINT. Story points vs tiempo. Ve si llegamos a tiempo.
- **Epic Burndown:** progreso de una épica a lo largo de sprints.
- **Velocity Chart:** trabajo completado por sprint vs compromiso. Mide capacidad sostenida.

3 Cs de historias de usuario

- **Card:** describe la intención en una tarjeta.
- **Conversation:** los interesados comunican detalles y requisitos.
- **Confirmation:** se definen criterios de aceptación.

7. Product Discovery (MVP y Dual Track)

Disciplina para construir mejores productos. Busca que el producto resuelva un problema real.

MVP (Minimum Viable Product)

- Versión más simple del producto.
- Valida la idea con el menor esfuerzo posible.
- Solo incluye funcionalidades indispensables.

Evolución del MVP

- **ETP (Earliest Testable Product):** testeable, no funciona de verdad.
- **EUP (Earliest Usable Product):** ya es funcional.
- **ELP (Earliest Lovable Product):** el producto que los usuarios podrían amar.

Discovery como Dual Track

El discovery NO solo se da al inicio, es paralelo al desarrollo. A la par se exploran nuevas ideas.

Mentalidad del Product Discovery

- **Colaboración y aprendizaje en equipo.**
- **User Centered Design.**
- **Humildad intelectual:** solo hay hipótesis, no certezas.

Problema vs Solución

Se diverge en los problemas y se converge en las soluciones.

Visualizaciones enfocadas en el usuario

UX Personas

- Representa un grupo de usuarios con comportamientos, objetivos y motivaciones similares.
- Campos: datos, problemas, comportamiento, necesidades y objetivos.

Mapa de Empatía

- Piensa y siente / ve / dice y hace / escucha / dolores y necesidades.

8. Historias de Usuario

Técnica ágil que describe una funcionalidad desde la perspectiva del usuario.

"Como <rol>, quiero <funcionalidad>, para <beneficio/motivación>."

Framework QUS (Quality User Story)

- **Calidad sintáctica:** fácil de entender, atómica, formato correcto.
- **Calidad semántica:** conceptos no ambiguos. Expresa el problema, no la solución.
- **Calidad pragmática:** no se repiten, estimables, completas, independientes.

Criterios INVEST

- **Independent:** no se repiten, paralelizables.
- **Negotiable:** se ajustan según necesidad.
- **Valuable:** aporta valor al usuario.
- **Estimable:** se puede medir el esfuerzo.
- **Small:** pequeña, atómica, desarrollable en una iteración.
- **Testable:** se puede verificar que está bien hecha.

No se recomienda

- Que la historia se centre en la solución.
- Que presuponga detalles de implementación.
- Meter múltiples oportunidades en una sola historia.

Cómo encontrar historias

User Story Mapping

- Agrupa por funcionalidades.
- Actividades rojas: funcionalidades principales.
- Amarillas: pasos de cada funcionalidad.
- Azules: tareas técnicas versionadas.

Impact Mapping

- Orientado a objetivos del negocio.
- Objetivo → Actores → Impactos → Entregables.

Criterios de Aceptación

"Dado que <contexto>, cuando <evento>, entonces <consecuencia>."

- Claros, concisos, verificables, independientes, orientados al problema.

BDD (Behavior Driven Development)

- Extensión de TDD.

- Define comportamiento ANTES de implementar.
- Ciclo: User Story → Discovery → Formulation → Automation.

Casos de Uso

- Acciones del sistema que generan resultado de valor para un actor.
- Escenario principal + escenarios alternativos.
- Formal, estructurado, menos iterativo que user stories.

9. Modelo de Dominio

Complejidad del desarrollo

complejidad = complejidad esencial + complejidad accidental

complejidad esencial = complejidad del problema + complejidad de la solución

Mecanismos POO para atacar complejidad: descomposición, abstracción, jerarquías.

Modelos

- **Modelo de dominio:** representación conceptual del problema. Define entidades y reglas de negocio.
- **Modelo de diseño:** agrega detalles técnicos (clases controladoras, patrones de diseño, dependencias).

Análisis Sintáctico

Obtiene el modelo de dominio extrayendo de textos:

- Actores (humanos).
- Objetos físicos.
- Lugares.
- Eventos.
- Procesos.

CRUD

Create, Read, Update, Delete. Si falta una operación, probablemente falta una funcionalidad.

Patrones de Colaboración

1. Genérico-Específico

- **Actor-Rol:** un actor conoce múltiples roles pero juega uno a la vez.
- **Item-Item específico:** el ítem tiene info común; la variante especializa.

2. Entero-Parte

- **Gran lugar-Lugar, Ensamble-Parte, Contenedor-Contenido, Grupo-Miembro.**

3. Específico-Transacción

- **Transacciones simples:** Rol + Lugar + Ítem específico.
- **Transacciones complejas:** Transacción compuesta + Line Items.
- **Transacción cronológica:** relaciona dos transacciones, la T sucede antes de la TCr.

Reglas de negocio

- **Tipo:** medicamento solo en container refrigerado.
- **Multiplicidad:** pallet hasta 10 cajas.
- **Propiedad:** pago con número de tarjeta válido.
- **Estado:** orden cancelada no se entrega.

- **Conflicto:** alcohol + menor de edad.

10. UML

Formalización de aspectos del software en diagramas. Estructura y comportamiento.

Estereotipos <<>>

Mecanismo de extensión. Modifica comportamiento.

En diagramas de componentes

- <<component>>: módulo lógico reusable.
- <<executable>>: componente ejecutable.

En diagramas de despliegue

- <<node>>: recurso computacional.
- <<device>>: hardware concreto.
- <<execution environment>>: entorno lógico (browser, node.js, mongodb).

En diagramas de clase

- <<interface>>, <<datatype>>, <<enumeration>>, <<stereotype>>.

Diagramas de Estructura

- **Objetos:** instancias concretas.
- **Clases:** atributos, operaciones, relaciones.
- **Paquetes:** mala práctica = agrupar por capa técnica (controller/service/repository). Buena = agrupar por funcionalidad de negocio.
- **Componentes:** rectángulos, interfaces (semicírculos), dependencias (flechas).
- **Despliegue:** Device → Execution Environment → Component. Muestra protocolos (HTTPS, TCP).
- **Perfil UML:** extensión del metamodelo para definir estereotipos personalizados.

Diagramas de Comportamiento

- **Actividad:** proceso de negocio. Círculo negro=inicio, rectángulo redondeado=acción, hexágono=decisión, fork/join=barras.
- **Casos de uso:** funcionalidades desde el usuario.
- **Máquina de estados:** estados y eventos que los cambian.
- **Secuencia:** quién habla con quién en el tiempo.
- **Comunicación:** relación entre objetos, sin foco temporal.
- **Tiempos:** evolución a lo largo del tiempo.

11. Atributos de Calidad

Propiedad testeable que indica cómo el sistema satisface las necesidades de los stakeholders.

Clasificación según cómo se evalúan

- **Internos:** sin ejecutar el sistema (código, arquitectura, docs).
- **Externos:** solo ejecutando. Punto de vista del usuario.

Clasificación según comportamiento

- **Operacionales:** durante el uso diario.
- **No-Operacionales:** durante mantenimiento y desarrollo.

Por qué se rediseñan los sistemas

Por violar atributos de calidad: mantenimiento, portabilidad, escalabilidad, performance, seguridad, interfaces obsoletas.

Conflictos entre atributos

- **Seguridad vs Confiabilidad:** ante ataque conviene detener sistema (↓ disponibilidad).
- **Portabilidad vs Performance:** optimizaciones específicas rompen portabilidad.
- **Seguridad vs Usabilidad:** validaciones rigurosas dificultan el acceso.
- **Performance vs Modificabilidad:** sistema acoplado con hacks es más performante.

Principales atributos

- **Disponibilidad:** % tiempo operativo. Mejora con balanceo, réplica, respaldo, escalabilidad.
- **Performance:** latencia + capacidad (eventos/tiempo).
- **Interoperabilidad:** integración con otros sistemas (interfaces, protocolos, APIs).
- **Usabilidad:** comprensible, fácil, intuitiva, atractiva.
- **Seguridad:** DoS, auth, anti-inyección, encriptación.
- **Escalabilidad:** Vertical (más recursos) u Horizontal (más máquinas).

12. Arquitectura de un Sistema

Definición

- **IEEE-1471**: propiedades fundamentales: elementos, relaciones, principios de diseño y evolución.
- **Martin Fowler**: decisiones importantes y difíciles de cambiar.

Modelo de Vistas 4+1 (Kruchten)

Vista Lógica

- Requisitos funcionales.
- Abstracción, encapsulamiento, herencia.
- Para desarrolladores.

Vista de Desarrollo (Componentes)

- Organización real de módulos.
- Empaquetamiento en bibliotecas/subsistemas.

Vista de Procesos

- Requisitos no funcionales: rendimiento, disponibilidad, concurrencia.
- Agrupación de tareas ejecutables, replicables para distribuir carga.

Vista Física (Despliegue)

- Disponibilidad, confiabilidad, rendimiento, escalabilidad.
- Mapeo a nodos. Diferentes configuraciones (dev, test, prod).

Vista de Escenarios (+1)

- Casos de uso más importantes.
- Unifica las 4 vistas. Guía descubrimiento, pruebas y validación.

13. Patrones de Arquitectura de Software

Layers (Capas)

- Cada capa con responsabilidad específica.
- Se comunica solo con capas adyacentes.

Ejemplo backend moderno: REST Controller → Services → Domain → Persistence (con interface Repository).

Broker

- Intermediario coordina comunicación.
- Servidores no se conocen entre sí, solo al broker.

Pipe & Filter

- Filtros/mapeos secuenciales.
- Ejemplos: HTTP filters, Unix commands, compilers.

Hexagonal Architecture

- Dos partes: reglas de negocio (núcleo) y periférico.
- El núcleo NO depende de detalles técnicos.
- Puertos y adaptadores.

Onion / Clean Architecture

Regla: las dependencias apuntan hacia el centro.

- **Entities:** reglas de empresa. NO DEPENDEN DE NADA.
- **Use Cases:** reglas de aplicación. SOLO DEPENDEN DE ENTIDADES.
- **Interface Adapters:** controllers, presenters, gateways.
- **Frameworks & Drivers:** UI, DB, devices, ORM. Más externo.

Microservicios

- Aplicación descompuesta en servicios autónomos.
- Comunicación por HTTP o mensajería asíncrona.

Micro Frontends

- Cada uno: funcionalidad autónoma del frontend.
- Se componen en una app contenedora.

14. RESTful API

REST

- Estilo de arquitectura para servicios web.
- Representational State Transfer.
- Usa HTTP.

NO es

- Un estándar ni un protocolo.
- Un reemplazo de SOAP.
- Una biblioteca.

Características

Cliente-servidor + Stateless

- Cada request es independiente.
- Contiene TODA la info necesaria.

Cacheable

- Metadatos: Expires, Cache-Control (max-age), Last-Modified.

Compresión

- Accept-Encoding (cliente) / Content-Encoding (server).

URIs

- Semánticas, sustantivos en plural.
- Principales (/users) y subordinados (/users/{id}/addresses).

Verbos HTTP

- **GET:** obtener recurso.
- **POST:** crear recurso.
- **PUT:** reemplazar recurso completo.
- **PATCH:** modificación parcial (no pisa).
- **DELETE:** borrar recurso.
- **OPTIONS:** describir opciones.

Status Codes

Rango	Código	Significado
2xx Success	200	OK
	201	Created
3xx Redirect	3xx	Redirection

Rango	Código	Significado
4xx Client Error	400	Bad Request
	401	Autorización requerida
	404	Not Found
	408	Request Timeout
	409	Conflict
5xx Server Error	500	Internal Server Error
	502	Bad Gateway
	504	Gateway Timeout

HATEOAS

Las respuestas contienen hipervínculos a recursos relacionados. El cliente descubre recursos dinámicamente.

Security Design Principles

Principio	Descripción
Least Privilege	Exigir el menor privilegio
Fail-Safe Defaults	Por defecto recursos NO accesibles
Complete Mediation	Validar completamente todos los accesos
Keep It Simple	Diseño simple
HTTPS	Encriptación en protocolo
Password Hashes	Nunca en texto plano
No info en URLs	Usar headers o body
Timestamp	Prevenir ataques de repetición
Validar parámetros	Prevenir inyecciones
Monitoreo	Detectar patrones inusuales

Autenticación vs Autorización

- **Autenticación:** verifica tu identidad.
- **Autorización:** qué podés hacer y a qué accedés.

Mecanismos de Autenticación

Basic Auth

- user:password en Base64 (encodeado, NO encriptado).

Authorization: Basic <base64(user:pass)>

API Keys

- Token secreto en cada request.

X-API-Key: <key>

- NUNCA en URL.

Bearer Authentication (Token)

Authorization: Bearer <token>

- **JWT:** info codificada, validable SIN consultar DB.
- **Opaque Token:** string aleatoria, requiere DB.

JWT

Formato: Header.payload.signature

- Header: algoritmo y tipo (encodeado base64).
- Payload: metadata (uid, role, etc). Encodeado base64.
- Signature: hash(header + '.' + payload + clave privada). CIFRADO.

Propiedades

- Si solo está firmado: legible pero inalterable.
- Stateless: no se consulta DB para validar.

Refresh Token

Permite renovar el access token sin re-pedir credenciales.

Versionado

- **URI:** /api/v1/resource
- **Header custom:** Accept-version: v1
- **Accept:** application/vnd.example.v1+json

15. Sprint Planning, Daily y Retros — ampliado

Durante el Sprint Planning

- Si una historia es demasiado grande, se divide en tareas más pequeñas.
- Se decide la versión más simple para entregar valor temprano.
- Si hay dudas, el equipo define SUPOSICIONES (ej: "asumimos que no se pide CAPTCHA en esta versión").
- Se estima solo cuando hay suficiente claridad.

Participan

- Scrum Master, Developers, Product Owner.

Qué se define

- Qué tareas del backlog atacar (capacidad y velocidad del equipo).
- Objetivo del sprint (qué queremos lograr y por qué).
- Plan inicial de cómo realizar las tareas.

Resultado del Sprint Planning

- 1. Sprint Goal (a dónde nos orientamos todos).
- 2. Sprint Backlog completo.
- 3. Plan inicial.
- 4. Noción de impedimentos del equipo.

Daily Scrum — detalle

- Reunión diaria, máx 15 minutos.
- Participa dev team + scrum master (PO opcional).

Las 3 preguntas clave

- ¿Qué hice ayer que ayudó al equipo a alcanzar el objetivo del sprint?
- ¿En qué trabajaré hoy para contribuir al objetivo común?
- ¿Algo me bloquea a mí o a mi compañero?

Buenas prácticas

- Mantener dentro de 15 min (responsabilidad del Scrum Master).
- Hablar de impedimentos tangibles y buscar ayuda si hace falta.
- NO resolver los problemas ahí (se hace después).
- Usar herramientas visuales (Kanban/Scrum board) para sincronizar.
- Si hay invitados, el master evita que interrumpen.
- NO son reportes individuales.

Retros — detalle

- Se obtiene feedback y se corrige el rumbo.

Tips

- Asumir que todos pusieron su mejor esfuerzo.
- Venir preparado y fomentar participación.
- Se escucha, NO se defiende.
- Hablar de problemas SOLUCIONABLES con acciones concretas.

Ejemplos de buenos/malos temas

- **Mal tema:** "el login tiene un bug" (es un issue técnico, no de proceso).
- **Buen tema:** aspectos positivos del equipo, falta de comunicación con el cliente, etc.

16. Headers, Query Strings y Request Bodies en REST

Headers principales

Compresión

Accept-Encoding: gzip (request, cliente)

Content-Encoding: gzip (response, server)

Encoding / Content-Type

Content-Type: application/json (response, server)

Cacheabilidad

Expires: Fri, 19 Nov 2021 19:20 EST

Cache-Control: max-age=3600

Last-Modified: Fri, 19 May 2021 09:17:49 EST

Autenticación

Authorization: Basic <base64(user:pass)>

X-API-Key: <key> (puede ser aleatoria o JWT)

Authorization: Bearer <token JWT>

Versionado

Accept: application/vnd.example.v1+json

Accept-version-custom: v1

Paginado (response)

Pagination-Count: 100 (total de elementos)

Pagination-Page: 5 (número de página)

Pagination-Limit: 20 (elementos por página)

Query Strings (URL)

Filtrado

GET /users?age=18&country=AR&active=true

- Parámetros dinámicos definidos por el cliente.
- Recomendado para GET.

Ordenamiento

GET /events?sort=criterio1,-criterio2

- Usa sort= con los campos.
- El guión '-' indica orden descendente.
- Separar por comas para múltiples criterios.

Versionado en URL

GET [url]/api/v1/resource

Request Body

Se sigue un estándar como JSON:API que define los campos:

- **data:** recursos que el cliente solicita/recibe.
- **errors:** reemplaza data cuando hay errores.
- **metadata:** timestamp, paginado.
- **links:** URLs de interés (HATEOAS).

Filtrado/orden/paginado en body

Quality	Label en body	Emisor
Filtrado	"filters": { ... }	Client (POST, no estandarizado)
Ordenamiento	"sort": [...]	Client (POST, no estandarizado)
Paginado	"meta": {page, per_page, page_count, total_count}	Server response (estandarizado)

Ejemplo didáctico: login

Descripción: Permite que los usuarios inicien sesión.

Request:

POST /api/v1/users/token

Authorization: Basic <base64(email:password)>

Response exitosa (200 OK):

```
{ "access_token": "eyJhbG...", "refresh_token": "m9obi..." }
```

Response fallida (401 Unauthorized):

Si las credenciales son incorrectas.

Nota: en la renovación de token se usa Authorization: Bearer <refresh_token> con el mismo endpoint POST /api/v1/users/token.

Tips finales para el parcial

- Saber diferenciar Output / Outcome / Impacto con ejemplos concretos.
- Todos los SOLID con ejemplo corto.
- Historia de usuario vs Caso de uso, Burndown vs Velocity Chart.
- INVEST + QUS + 3 Cs.
- Modelo de dominio: análisis sintáctico → CRUD → patrones de colaboración → reglas de negocio.
- 4+1 vistas con su público objetivo.
- Clean Architecture: dependencias hacia adentro.
- REST: stateless, cacheable, URIs en plural, HATEOAS.
- JWT: estructura y cómo mantiene el server stateless.
- Headers típicos: Authorization, Accept, Cache-Control, Pagination-.*.
- Conflictos entre atributos de calidad (seguridad vs usabilidad, etc.).
- En el sprint planning: capacidad, objetivo del sprint, plan inicial.
- En retros: temas solucionables, no issues técnicos puntuales.

¡Éxitos en el parcial, Tomas! 🍀

Ingenieria de Software 1

Resumen Completo — Primer Parcial

Version 3 — Ampliada con Practica y Ejercicios

UBA — Ingenieria Informatica

Tomas

2025

Indice — Contenido Practico Agregado

A	SOLID y Diseno de Clases	Ejercicios con escenarios, diagramas de clases, errores comunes
B	Diagramas UML	Secuencia, actividad, casos de uso, clases — con escenarios tipo examen
C	Scrum e Historias de Usuario	Escritura de HU, criterios de aceptacion, planning, retros
D	Arquitectura de Software	Como elegir y justificar: Layers, Hexagonal, Clean, Microservicios
E	Guia para encarar un examen	Como leer un escenario y decidir que aplicar

A — SOLID y Diseño de Clases (Practica)

En el examen te van a dar un escenario con clases mal diseñadas y te van a pedir identificar que principio se viola, como corregirlo, o directamente diseñar desde cero respetando SOLID.

A.1 — Single Responsibility Principle (SRP)

Ejercicio 1 Identificar violacion de SRP

Escenario: Tienes la clase 'Factura' que: calcula el total, genera el PDF, envia el email al cliente y guarda en base de datos. El examinador pregunta: ¿Que principio se viola y como lo corrigis?

Analisis:

- La clase tiene 4 responsabilidades distintas.
- Si cambia la logica de email, tengo que tocar la clase Factura — no tiene sentido.
- Si cambia el formato del PDF, tambien tengo que tocar Factura.

Solucion: Separar en: Factura (calculo del total), FacturaPDFGenerator, EmailService, FacturaRepository. Cada clase tiene un solo motivo para cambiar.

Tip de examen: Pregunta clave para detectar SRP: ¿cuantas razones tiene esta clase para cambiar? Si la respuesta es mas de 1, viola SRP.

A.2 — Open/Closed Principle (OCP)

Ejercicio 2 Diseño extensible sin modificar código existente

Escenario: Un sistema calcula descuentos: si el cliente es VIP descuenta 20%, si es estudiante 10%, si es normal 0%. El código usa if/else. Agregar un nuevo tipo 'empleado' requiere modificar la clase. ¿Como lo mejorias?

El problema (código malo):

```
class CalculadorDescuento:
    def calcular(self, tipo_cliente, monto):
        if tipo_cliente == 'VIP':
            return monto * 0.80
        elif tipo_cliente == 'estudiante':
            return monto * 0.90
        # Agregar empleado = MODIFICAR esta clase
```

La solución (OCP aplicado):

```
from abc import ABC, abstractmethod
class EstrategiaDescuento(ABC):
    @abstractmethod
    def calcular(self, monto):
        pass
class ClienteVIP(EstrategiaDescuento):
    def calcular(self, monto):
        return monto * 0.80
class ClienteEstudiante(EstrategiaDescuento):
    def calcular(self, monto):
        return monto * 0.90
# Agregar empleado = nueva clase, NO modifiko nada existente
class ClienteEmpleado(EstrategiaDescuento):
    def calcular(self, monto):
        return monto * 0.85
```


Tip de examen: OCP se implementa casi siempre con polimorfismo o el patron Strategy/Template Method. Cuando veas un if/else grande que crece, es senal de violacion OCP.

A.3 — Liskov Substitution Principle (LSP)

Ejercicio 3 Detectar herencia incorrecta

Escenario: Tienes una clase 'Ave' con metodo volar(). La clase 'Pinguino' hereda de Ave pero lanza una excepcion en volar() porque los pingüinos no vuelan. ¿Que principio viola y como lo arreglas?

Analisis:

- Un Pinguino NO puede sustituir a un Ave en cualquier contexto — LSP violado.
- Si alguien llama ave.volar() esperando que funcione, Pinguino rompe esa expectativa.

Solucion: Separar jerarquia: Ave (comportamientos comunes), AveVoladora extends Ave (agrega volar()), Pinguino extends Ave (sin volar()). Alternativa: extraer interfaz 'Volador' solo para quien vuela.

Error comun: Error comun: pensar que si B 'es un tipo de' A en el mundo real, entonces B debe heredar de A en codigo. LSP requiere sustituibilidad de comportamiento, no solo de concepto.

A.4 — Interface Segregation Principle (ISP)

Ejercicio 4 Interfaces demasiado gordas

Escenario: Interface 'Trabajador' tiene: trabajar(), comer(), dormir(). La clase 'Robot' implementa Trabajador pero comer() y dormir() no tienen sentido para un robot. ¿Como lo corriges?

Solucion: Separar en interfaces especificas: Trabajador (trabajar()), SerVivo (comer(), dormir()). Robot solo implementa Trabajador. Un HumanoTrabajador implementa ambas.

Tip de examen: Senal de alarma ISP: una clase implementa una interfaz y deja metodos vacios o con 'throw NotImplemented'. Eso indica que la interfaz es demasiado amplia.

A.5 — Dependency Inversion Principle (DIP)

Ejercicio 5 Dependencia de clases concretas

Escenario: La clase 'PedidoService' instancia directamente 'MySQLPedidoRepository' en su constructor. Si queremos cambiar a PostgreSQL o usar un mock en tests, hay que modificar PedidoService. ¿Como aplicarías DIP?

```
# MAL: dependencia directa a concrecion class PedidoService: def __init__(self): self.repo = MySQLPedidoRepository() # acoplado! # BIEN: depender de abstraccion class IPedidoRepository(ABC): @abstractmethod def guardar(self, pedido): pass class PedidoService: def __init__(self, repo: IPedidoRepository): # inyeccion self.repo = repo # En produccion: service = PedidoService(MySQLPedidoRepository()) # En tests: service = PedidoService(MockPedidoRepository())
```

Tip de examen: DIP + inyeccion de dependencias van casi siempre de la mano. Si ves 'new ClaseCorrectaX()' dentro de otra clase, hay violacion de DIP.

Tabla Resumen — SOLID de un vistazo

SRP	¿Cuántas razones para cambiar?	Clase con múltiples responsabilidades	Separar clases
OCP	¿Tengo que modificar para extender?	if/else que crece con cada tipo nuevo	Polimorfismo / Strategy
LSP	¿La subclase rompe comportamiento del padre?	Excepción o método vacío en override	Reestructurar jerarquía
ISP	¿La clase implementa métodos que no usa?	NotImplemented / métodos vacíos	Interfaces específicas
DIP	¿Dependo de una clase concreta?	'new ConcreteClass()' dentro de otra clase	Inyección de dependencias

B — Diagramas UML (Practica)

En el examen te dan un escenario y te piden uno o mas diagramas. Lo clave es saber **que diagrama elegir**, **que elementos incluir** y **que omitir**. A continuacion: guias con escenarios tipo parcial.

B.1 — Diagrama de Casos de Uso

Cuando lo piden: cuando quieren ver las funcionalidades del sistema desde la perspectiva de los actores. Responde '¿que puede hacer cada actor?'

Ejercicio 6 Sistema de turnos medicos

Escenario: Un paciente puede registrarse, buscar turnos disponibles y sacar turno. Un medico puede ver su agenda y cancelar turnos. El sistema envia notificaciones automaticamente al confirmar un turno.

Como resolverlo — pasos:

- **1. Identificar actores:** Paciente, Medico, Sistema (actor secundario)
- **2. Identificar casos de uso:** Registrarse, Buscar turnos, Sacar turno, Ver agenda, Cancelar turno, Enviar notificacion
- **3. Relacionar:** <<include>> o <<extend>> si corresponde

Relacion	Significado	Ejemplo en el escenario
<<include>>	Siempre se ejecuta como parte del CU base	Sacar turno <<include>> Enviar notificacion
<<extend>>	Se ejecuta condicionalmente (extension opcional)	Cancelar turno <<extend>> Notificar cancelacion

Tip de examen: <<include>> = obligatorio (siempre pasa). <<extend>> = opcional o condicional (puede pasar). El error mas comun es confundirlos.

B.2 — Diagrama de Secuencia

Cuando lo piden: cuando quieren ver el flujo de mensajes entre objetos/capas para un caso de uso especifico. Responde '¿quien le habla a quien y en que orden?'

Ejercicio 7 Login de usuario con JWT

Escenario: Modelar el flujo de login: el cliente envia credenciales, el controller las valida, consulta al servicio de autenticacion, este verifica en la base de datos, y si es correcto retorna un JWT al cliente.

Participantes (lifelines):

- Cliente (actor externo)
- AuthController

- AuthService
- UserRepository
- JWTUtil

Flujo de mensajes a representar:

- 1. Cliente -> AuthController: POST /login {email, password}
- 2. AuthController -> AuthService: autenticar(email, password)
- 3. AuthService -> UserRepository: findByEmail(email)
- 4. UserRepository --> AuthService: User | null
- 5. AuthService: verifica hash de password
- 6. AuthService -> JWTUtil: generarToken(userId, rol)
- 7. JWTUtil --> AuthService: token
- 8. AuthService --> AuthController: token
- 9. AuthController --> Cliente: 200 OK { access_token, refresh_token }

Tip de examen: En secuencia: las flechas solidas son llamadas, las punteadas son retornos. Los 'alt/opt/loop' son fragmentos combinados — usalos si el examen pide modelar el caso de error tambien.

Fragmento 'alt' para el caso de error:

- alt [credenciales validas]: flujo normal (arriba)
- alt [credenciales invalidas]: AuthController --> Cliente: 401 Unauthorized

B.3 — Diagrama de Clases

Cuando lo piden: para modelar estructura estatica — clases, atributos, metodos y relaciones entre ellas.

Ejercicio 8 Sistema de e-commerce

Escenario: Modelar: un Cliente puede realizar varios Pedidos. Cada Pedido tiene uno o mas ItemPedido. Cada ItemPedido referencia un Producto. Un Pedido tiene un estado (PENDIENTE, CONFIRMADO, ENVIADO). El pago puede ser con Tarjeta o Transferencia.

Guia para construirlo:

- **1. Identificar clases:** Cliente, Pedido, ItemPedido, Producto, Pago, PagoTarjeta, PagoTransferencia
- **2. Atributos clave:** Pedido(id, fecha, estado: EstadoPedido), Producto(id, nombre, precio), ItemPedido(cantidad, precioUnitario)
- **3. Relaciones:**
 - Cliente 1 ----- * Pedido (asociacion, un cliente tiene muchos pedidos)
 - Pedido 1 ----- + ItemPedido (composicion, si borro pedido se borran los items)
 - ItemPedido * ----- 1 Producto (asociacion, muchos items pueden referenciar 1 producto)
 - Pedido 1 ----- 1 Pago (asociacion)
 - Pago <|-- PagoTarjeta, PagoTarjeta <|-- PagoTransferencia (herencia)
- **4. Enum:** EstadoPedido { PENDIENTE, CONFIRMADO, ENVIADO, CANCELADO }

Relacion	Notacion	Significado	Ejemplo
Asociacion	——>	Un objeto conoce a otro	Cliente conoce Pedido

Agregacion		Todo-parte, parte puede existir sola	Universidad tiene Profesores
Composicion		Todo-parte, parte NO existe sin el todo	Pedido tiene ItemPedido
Herencia		Subclase extiende superclase	PagoTarjeta extends Pago
Realizacion		Clase implementa interfaz	MySQLRepo implements IRepo
Dependencia		Uso temporal (parametro o variable local)	Service usa DTO en un metodo

B.4 — Diagrama de Actividad

Cuando lo piden: para modelar flujos de trabajo o procesos de negocio. Responde '¿que pasos sigue el proceso?'

Ejercicio 9 Proceso de compra en e-commerce

Escenario: Modelar el proceso: el usuario agrega productos al carrito, procede al checkout, el sistema verifica stock, si hay stock se procesa el pago, si el pago es exitoso se confirma el pedido y se envia notificacion, si falla el pago se muestra error.

Elementos a usar:

Elemento	Simbolo	Uso en este caso
Inicio	Circulo negro relleno	Inicio del proceso
Fin	Circulo negro con borde	Pedido confirmado / Error mostrado
Accion	Rectangulo redondeado	'Verificar stock', 'Procesar pago'
Decision	Rombo / Hexagono	[hay stock?] [pago exitoso?]
Fork/Join	Barra gruesa	Si confirmas pedido Y envias notif en paralelo
Swim lanes	Columnas por actor	Separar: Usuario / Sistema / Banco

Tip de examen: Si el enunciado menciona 'en paralelo' o 'a la vez', usar Fork/Join. Si menciona 'si/sino' o condicion, usar Decision (rombo).

C — Scrum e Historias de Usuario (Practica)

C.1 — Escribir Historias de Usuario correctamente

El examen te da una funcionalidad y te pide escribir la HU con criterios de aceptacion. Tambien puede darte una HU mal escrita y pedirte que la corrijas.

Ejercicio 10 Corregir una HU mal escrita

Escenario: HU original (mal escrita): 'Como sistema, quiero que la base de datos guarde los usuarios con SHA256 para que el password este seguro.'

Problemas:

- El actor es 'sistema' — las HU deben tener un actor humano o rol de negocio.
- Presupone la solucion tecnica (SHA256) — una HU describe el QUE, no el COMO.
- No expresa el beneficio real para el usuario.

Solucion: HU corregida: 'Como usuario registrado, quiero que mi contrasena este protegida, para que mis datos esten seguros si alguien accede a la base de datos.'

Ejercicio 11 Escribir HU con criterios de aceptacion

Escenario: Funcionalidad: el usuario puede buscar productos por nombre. Los resultados se muestran paginados de a 10. Si no hay resultados, se muestra un mensaje.

Historia de Usuario:

Como comprador, quiero buscar productos por nombre, para encontrar rapidamente lo que necesito sin navegar por todo el catalogo.

Criterios de Aceptacion (formato Gherkin):

Escenario 1: Búsqueda con resultados Dado que estoy en la pagina de búsqueda Cuando escribo 'zapatillas' en el buscador Entonces veo hasta 10 productos que contienen 'zapatillas' en el nombre Y puedo navegar a la siguiente pagina de resultados Escenario 2: Búsqueda sin resultados Dado que estoy en la pagina de búsqueda Cuando escribo 'xyzabc123' en el buscador Entonces veo el mensaje 'No encontramos productos para tu búsqueda'

C.2 — Verificar criterios INVEST

Ejercicio 12 Evaluar si una HU cumple INVEST

Escenario: HU: 'Como administrador, quiero poder gestionar el sistema completo de usuarios, incluyendo alta, baja, modificacion, bloqueo, reportes y exportacion a Excel.'

Criterio	Cumple?	Por que?
Independent	SI	No depende de otras HU
Negotiable	SI	Se puede ajustar el alcance
Valuable	SI	Aporta valor al admin
Estimable	NO	Demasiado grande para estimar con precision
Small	NO	Es una epica: 6+ funcionalidades distintas
Testable	Parcial	Sin criterios de aceptacion definidos

Solucion: Esta HU es en realidad una Epica. Hay que dividirla: 'Alta de usuario', 'Baja de usuario', 'Modificacion de usuario', 'Bloqueo de cuenta', 'Reportes de usuarios', 'Exportacion a Excel' — cada una como HU independiente.

C.3 — Sprint Planning: como responder preguntas tipicas

Ejercicio 13 Organizar un Sprint Planning

Escenario: El equipo tiene velocidad de 30 story points por sprint. El backlog priorizado tiene: HU-1 (8pts), HU-2 (13pts), HU-3 (5pts), HU-4 (8pts), HU-5 (3pts). ¿Que historias entran en el sprint? ¿Que pasa si HU-3 es muy grande para estimarla bien?

Respuesta:

- Entran: HU-1 (8) + HU-2 (13) + HU-3 (5) + HU-4 (8) = 34 pts — excede por 4.
- Ajuste: HU-1 + HU-2 + HU-3 + HU-5 = 29 pts — entra con 1 punto de margen.
- Si HU-3 no esta bien definida: el equipo anota una SUPOSICION y la refina antes de commitear.

Resultado del Sprint Planning (lo que siempre hay que mencionar):

- 1. Sprint Goal: descripcion de lo que se quiere lograr y por que
- 2. Sprint Backlog: lista de HU comprometidas con sus tareas
- 3. Plan inicial: como se van a abordar las tareas
- 4. Impedimentos detectados

C.4 — Retro: temas validos vs invalidos

Tema	Valido en Retro?	Por que?
'El login tiene un bug'	NO	Es un issue tecnico, va al backlog, no a la retro
'No tuvimos comunicacion con el PO durante el sprint'	SI	Problema de proceso/comunicacion, solucionable con acciones

'Las estimaciones fueron muy imprecisas'	SI	Aspecto del proceso del equipo, mejorable
'El servidor de produccion estuvo caido'	NO	Problema de infraestructura, no es un problema de proceso del equipo
'El equipo colaboro muy bien en pair programming'	SI	Positivo del proceso — reforzarlo

D — Arquitectura de Software (Practica)

En el examen pueden pedirte justificar que patron arquitectonico usarias dado un escenario, o diagramar la arquitectura de un sistema. La clave es saber los trade-offs de cada patron.

D.1 — Tabla comparativa de patrones

Patron	Ideal cuando...	Ventajas	Desventajas
Layers	App monolitica simple, equipo chico, CRUD basico	Simple, facil de entender, bajo overhead	Dificil de escalar partes, acoplamiento entre capas
Hexagonal	Necesitas testar la logica de negocio sin infraestructura	Testeable, intercambiable, desacoplada del framework	Mas compleja de configurar, mas indirecciones
Clean Arch.	Dominio complejo, logica de negocio critica	Alta mantenibilidad, las reglas de negocio no dependen de nada	Mucho boilerplate, curva de aprendizaje
Microservicios	Equipos grandes, necesidad de escalar partes independientemente	Escalabilidad por servicio, deploys independientes	Complejidad operacional alta, comunicacion por red
Broker	Multiples servicios que no deben conocerse entre si	Desacoplamiento, flexibilidad para agregar nuevos servicios	El broker es un punto unico de falla

D.2 — Ejercicio: elegir y justificar arquitectura

Ejercicio 14 Sistema de gestion hospitalaria

Escenario: Startup medica quiere construir un sistema para: gestionar turnos, historiales clinicos y facturacion. Equipo de 4 developers. Plazo inicial: 6 meses MVP. Hay mucha logica de negocio en reglas medicas (restricciones de medicamentos, validaciones clinicas). Se anticipa que a futuro puede crecer mucho.

Como responder en el examen — estructura recomendada:

- **1. Analizar restricciones:** equipo chico (4 devs), MVP en 6 meses, logica compleja, crecimiento futuro.
- **2. Descartar opciones:** Microservicios es prematuro para un equipo de 4 con timeline ajustado.
- **3. Elegir:** Monolito con Clean Architecture o Hexagonal.
- **4. Justificar:**
 - La logica medica compleja se beneficia de tener las reglas de negocio en el nucleo, aisladas del framework.
 - El equipo chico puede moverse rapido con un solo deployable.
 - Cuando el sistema crezca, los limites internos permiten extraer microservicios gradualmente.

Solucion: Arquitectura elegida: Monolito con Clean Architecture. Capas: Entities (reglas medicas), Use Cases (flujos de negocio), Interface Adapters (controllers REST, repositorios), Frameworks (DB, Spring/Flask). Las dependencias apuntan hacia adentro — el dominio nunca depende de la DB ni del framework.

D.3 — Clean Architecture: la regla de dependencias

Capa	Contiene	Depende de	Ejemplo
Entities	Reglas de negocio de la empresa	NADA (es el nucleo)	Clase Pedido con sus validaciones
Use Cases	Reglas de la aplicacion	Solo Entities	CrearPedidoUseCase, CancelarPedidoUseCase
Adapters	Controllers, Presenters, Gateways	Use Cases y Entities	PedidoController, PedidoRepositoryImpl
Frameworks	DB, UI, frameworks externos	Todo lo anterior	Spring Boot, SQLAlchemy, React

Error comun: El error mas comun: poner logica de negocio en el Controller o en el Repository. El Controller solo traduce HTTP a llamadas de Use Case. La logica va en Use Cases o Entities.

E — Guia para Encarar un Examen con Escenarios

Cuando el examen te da un caso y te pide modelar, analizar o recomendar, hay una estructura mental que te ayuda a no perderte.

Paso 1 — Leer el escenario y subrayar:

- **Actores:** quienes participan (humanos, sistemas externos)
- **Acciones / funcionalidades:** que hace cada actor
- **Restricciones:** reglas de negocio, condiciones
- **Palabras clave de proceso:** 'si', 'cuando', 'entonces', 'en paralelo'

Paso 2 — Identificar que te piden:

Te piden	Usar	Foco principal
'¿Que puede hacer el usuario?'	Casos de uso	Actores + CU + include/extend
'Modelar el flujo de mensajes'	Diagrama de Secuencia	Lifelines + mensajes + retornos + fragmentos alt/opt
'Modelar la estructura / clases'	Diagrama de Clases	Clases + relaciones + multiplicidades
'Modelar el proceso / flujo de trabajo'	Diagrama de Actividad	Acciones + decisiones + fork/join + swimlanes
'Recomendar arquitectura'	Analisis de trade-offs	Tamano equipo + complejidad dominio + escalabilidad
'Identificar violacion de principio'	SOLID	Pregunta clave de cada principio + solucion
'Escribir HU / criterios'	Scrum / HU	Formato Como/Quiero/Para + Gherkin Dado/Cuando/Entonces

Paso 3 — Checklist antes de entregar

[SOLID]	■ Identifique el principio violado y lo nombre correctamente
[SOLID]	■ Propuse una solucion concreta, no solo describi el problema
[UML Clases]	■ Puse multiplicidades en todas las relaciones (1, *, 0..1)
[UML Clases]	■ Diferencie composicion de agregacion donde corresponde

[UML Secuencia]	■ Los retornos son flechas punteadas, las llamadas son solidas
[UML Secuencia]	■ Use fragmento 'alt' para los casos de error si los hay
[UML CU]	■ No confundi include con extend
[UML Actividad]	■ Use fork/join si hay actividades en paralelo
[HU]	■ El actor es un rol de negocio, no 'el sistema'
[HU]	■ No presupuse detalles tecnicos en la HU
[HU]	■ Los criterios de aceptacion son verificables y usan Dado/Cuando/Entonces
[Arquitectura]	■ Justifique la eleccion con contexto (tamano equipo, complejidad, escala)
[Arquitectura]	■ Mencione los trade-offs de la opcion elegida

